

Module 3 Activity 1: Describing Data

Purpose:

Create describe objects with Python to return descriptive information about spatial data using a standalone Python editing environment.

Instructions:

This exercise will be using PyScripter. Any Python environment will work.

1. Start by incorporating a ***try*** statement
2. Import the ***arcpy*** module

NOTE: The ***sys*** module is also imported. This is because you are working with specific pathnames referencing data. Although the script will run without the ***sys*** module, it can act as a failsafe.

3. Set the workspace environment using a single backslash referencing the path name

*remember that using a single backslash also refers to a line continuation. Use the ***r*** in front of the path to reference a ***raw path***

4. Create a variable that will reference your spatial data called ***fcName1***. Keep it logical as we will be referencing more than one feature class.
5. Use the ***describe*** function to return the spatial reference of the feature class
6. Repeat steps **4 & 5** by accessing the following datasets while creating a new variable for each one:
 - ***Parcels***
 - ***Burglary***
 - ***CrimeSurface***
 - ***Crime2009Table***

Your code should look like the screenshot below:

```
Python
? ▾ □ ×

try:
    #Import Python modules
    import arcpy, sys
    from arcpy import env

    #Set the workspace environment using a single backslash
    env.workspace = r"C:\GEOS456\Data.gdb"

    #Create a variable and check the spatial reference for CityBoundaries using
    #the describe function
    fcName1 = "CityBoundaries"
    fcName1Properties = arcpy.Describe(fcName1)

    #Create a variable and check the spatial reference for Parcels using
    #the describe function
    fcName2 = "Parcels"
    fcName2Properties = arcpy.Describe(fcName2)

    #Create a variable and check the spatial reference for Burglary using
    #the describe function
    fcName3 = "Burglary"
    fcName3Properties = arcpy.Describe(fcName3)

    #Create a variable and check the spatial reference for CrimeSurface using
    #the describe function
    fcName4 = "CrimeSurface"
    fcName4Properties = arcpy.Describe(fcName4)

    #Create a variable and check the spatial reference for Crime2009Table using
    #the describe function
    fcName5 = "Crime2009Table"
    fcName5Properties = arcpy.Describe(fcName5)
```

7. Use the **print** statement to generate other results of the **describe** functions. A print statement will need to be used for each feature and each description. Follow the following format to generate a detailed description of each feature.
 - Use the **shapeType** property to return the geometry of the feature
 - Use the **catalogPath** property to return the path of the data

```
#Print the results
print (fcName1 + " " + "saptial reference is: " \
      + fcName1Properties.spatialReference.name)
print (fcName1 + " " + "shape is: " + fcName1Properties.shapeType)
print (fcName1 + " " + "full path is: " + fcName1Properties.catalogPath)
```

The above screenshot references each of the variables and the specific descriptions of each feature as well as some additional text. While this can take time, the benefits of producing results like this will provide a good level of detail in the results pictured below:

```
CityBoundaries saptial reference is: GCS_WGS_1984
CityBoundaries shape is: Polygon
CityBoundaries full path is: C:\GEOS456\Data.gdb\CityBoundaries
```

8. Repeat the process for each of the features

NOTE: A line continuation has been used for readability purposes. This is optional, but it is encouraged as it makes some blocks easier to read.

9. Close the **try** statement with an **except** clause
10. Add another **print** statement to print a custom message in the event that something doesn't go as planned

```
Python
#Print the results
print (fcName1 + " " + "saptial reference is: " \
+ fcName1Properties.spatialReference.name)
print (fcName1 + " " + "shape is: " + fcName1Properties.shapeType)
print (fcName1 + " " + "full path is: " + fcName1Properties.catalogPath)
print (fcName2 + " " + "Saptial Reference is: " \
+ fcName2Properties.spatialReference.name)
print (fcName2 + " " + "shape is: " + fcName2Properties.shapeType)
print (fcName2 + " " + "full path is: " + fcName2Properties.catalogPath)
print (fcName3 + " " + "Saptial Reference is: " + \
fcName3Properties.spatialReference.name)
print (fcName3 + " " + "shape is: " + fcName3Properties.shapeType)
print (fcName3 + " " + "full path is: " + fcName3Properties.catalogPath)
print (fcName4 + " " + "Saptial Reference is: " + \
fcName4Properties.spatialReference.name)
print (fcName4 + " " + "shape is: " + fcName4Properties.shapeType)
print (fcName4 + " " + "full path is: " + fcName4Properties.catalogPath)
print (fcName5 + " " + "Saptial Reference is: " + \
fcName5Properties.spatialReference.name)
print (fcName5 + " " + "shape is: " + fcName5Properties.shapeType)
print (fcName5 + " " + "full path is: " + fcName5Properties.catalogPath)

except:
    print ("Something couldn't be found - go back and check your code!")
```

The entire code should look like the screenshot below:

```
Python
? ▾ □ ×

try:
    #Import Python modules
    import arcpy, sys
    from arcpy import env

    #Set the workspace environment using a single backslash
    env.workspace = r"D:\GEOS456\Data.gdb"

    #Create a variable and check the spatial reference for CityBoundaries using
    #the describe function
    fcName1 = "CityBoundaries"
    fcName1Properties = arcpy.Describe(fcName1)

    #Create a variable and check the spatial reference for Parcels using
    #the describe function
    fcName2 = "Parcels"
    fcName2Properties = arcpy.Describe(fcName2)

    #Create a variable and check the spatial reference for Burglary using
    #the describe function
    fcName3 = "Burglary"
    fcName3Properties = arcpy.Describe(fcName3)

    #Create a variable and check the spatial reference for CrimeSurface using
    #the describe function
    fcName4 = "CrimeSurface"
    fcName4Properties = arcpy.Describe(fcName4)

    #Create a variable and check the spatial reference for Crime2009Table using
    #the describe function
    fcName5 = "Crime2009Table"
    fcName5Properties = arcpy.Describe(fcName5)

    #Print the results
    print (fcName1 + " " + "spatial reference is: " \
    + fcName1Properties.spatialReference.name)
    print (fcName1 + " " + "shape is: " + fcName1Properties.shapeType)
    print (fcName1 + " " + "full path is: " + fcName1Properties.catalogPath)
    print (fcName2 + " " + "Spatial Reference is: " \
    + fcName2Properties.spatialReference.name)
    print (fcName2 + " " + "shape is: " + fcName2Properties.shapeType)
    print (fcName2 + " " + "full path is: " + fcName2Properties.catalogPath)
    print (fcName3 + " " + "Spatial Reference is: " \
    + fcName3Properties.spatialReference.name)
    print (fcName3 + " " + "shape is: " + fcName3Properties.shapeType)
    print (fcName3 + " " + "full path is: " + fcName3Properties.catalogPath)
    print (fcName4 + " " + "Spatial Reference is: " \
    + fcName4Properties.spatialReference.name)
    print (fcName4 + " " + "shape is: " + fcName4Properties.shapeType)
    print (fcName4 + " " + "full path is: " + fcName4Properties.catalogPath)
    print (fcName5 + " " + "Spatial Reference is: " \
    + fcName5Properties.spatialReference.name)
    print (fcName5 + " " + "shape is: " + fcName5Properties.shapeType)
    print (fcName5 + " " + "full path is: " + fcName5Properties.catalogPath)

except:
    print ("Something couldn't be found - go back and check your code!")
```

11. Run your code
12. Examine your results in the Python Interpreter

```
Python
print (fcName4 +" "+ "shape is: " + fcName4Properties.shapeType)
print (fcName4 +" "+ "full path is: " + fcName4Properties.catalogPath)
print (fcName5 +" "+ "Saptial Reference is: " + \
fcName5Properties.spatialReference.name)
print (fcName5 +" "+ "shape is: " + fcName5Properties.shapeType)
print (fcName5 +" "+ "full path is: " + fcName5Properties.catalogPath)

except:
    print ("Something couldn't be found - go back and check your code!")
CityBoundaries saptial reference is: GCS_WGS_1984
CityBoundaries shape is: Polygon
CityBoundaries full path is: C:\GEOS456\Data.gdb\CityBoundaries
Parcels Saptial Reference is: NAD_1983_UTM_Zone_12N
Parcels shape is: Polygon
Parcels full path is: C:\GEOS456\Data.gdb\Parcels
Burglary Saptial Reference is: GCS_WGS_1984
Burglary shape is: Point
Burglary full path is: C:\GEOS456\Data.gdb\Burglary
CrimeSurface Saptial Reference is: GCS_WGS_1984
Something couldn't be found - go back and check your code!
```

The code worked, but you will notice that the print statement inside the **except** clause has appeared. Incorporating the **try/except** control structure is useful in this case as it will return a message that is useful to the reader as opposed to just generating an error message. The code returned all the results it could find, but it couldn't find information about two features, thus returning a message.

13. Examine your results in more detail. You will notice that descriptive information was not returned for the following:

- **CrimeSurface**
- **Crime2009Table**

Answer the following questions

1. Why was only limited information returned for the **CrimeSurface** and no information returned for the **Crime2009Table**?
2. How would you fix your code so that it would return information from question 1?

You have likely noticed that this code is long and repetitive. This entire process can be streamlined using a FOR loop without the need of the **try/except**. Use your skills to figure it out!